SIMPLE PROMPT: Real-Time Bingo Betting Game (MVP)

Paste the "ROLE & RULES" and "TECH STACK" sections first, then give ONE PHASE at a time. Wait until each phase's tests pass before the next.

ROLE & RULES

You are a senior full-stack engineer. Build a simple, secure, fair, real-time multiplayer Bingo game with real-money wagering. Keep it small and working, not over-engineered.

Non-negotiable rules:

1. **Server decides everything.** Draws, wins, balances and payouts are computed only on the server. Never trust client input.
2. **Money is stored as integers** (santim/cents). Never floats. Every balance change is written to an append-only ledger inside a DB transaction. Money endpoints are idempotent (idempotency key).
3. **Fair draws.** Use a cryptographically secure RNG (`crypto.randomInt` , never `Math.random`). Use commit-reveal: publish SHA256(serverSeed) before the round, reveal serverSeed after, so players can verify.
4. **Basic protection.** 18+ only, self-exclusion option, deposit limit, rate limiting, audit log. Add COMPLIANCE.md: the operator must get a proper gaming license before going live. Do not hardcode a country.
5. Each phase ends with: working code, passing tests, short README note. No TODO stubs in money, RNG, win-check or auth code.
6. Typed, clean code, small commits.

TECH STACK

- Single repo with `server/` and `client/` (no monorepo complexity)
- Server: Node.js 20 + TypeScript + Fastify + Socket.IO + zod validation
- Database: PostgreSQL + Prisma (migrations)
- Redis: only add later if you need to scale to multiple servers
- Client: React + Vite + TypeScript + Tailwind, mobile-first, Amharic (am) + English (en)
- Tests: Vitest + Supertest
- Docker + docker-compose (`docker compose up` runs everything), `.env.example` , `/health` endpoint

---

PHASE 1: Setup

Scaffold the repo, ESLint, Prettier, strict TypeScript, zod-validated env config, pino logging, docker-compose (Postgres + server + client), and a `/health` endpoint.

PHASE 2: Game Engine (pure TypeScript, no I/O, fully tested)

1. 75-ball card generator (B 1-15, I 16-30, N 31-45, G 46-60, O 61-75, center FREE). Option: a fixed pool of 100-400 cards players pick from.
2. Secure draw: Fisher-Yates shuffle with CSPRNG and no modulo bias.
3. Provably fair: serverSeed -> publish hash -> derive draw order with HMAC_SHA256(serverSeed, roundId) -> reveal after round. Write `verifyRound()`.
4. Win patterns as JSON data: any line, four corners, full house.
5. Claim validation: card + called numbers + pattern -> valid/invalid. Multiple simultaneous winners split the pot equally (remainder handled deterministically).
6. Tests: card validity/uniqueness, uniform draw distribution, verify round-trip.

PHASE 3: Database + Auth

Tables: `users` (username UNIQUE case-insensitive, password_hash, phone optional, dob, role, status), `wallets` (balance_minor with CHECK >= 0), `ledger_entries` (append-only, idempotency_key UNIQUE), `rooms`, `rounds`, `round_draws`, `tickets`, `payouts`, `deposits`, `withdrawals`, `self_exclusions`, `audit_logs`.
Auth (username + password):

- **Register:** the player chooses a `username` (3-20 chars, letters/numbers/underscore, unique) and a `password` (min 8 chars), plus date of birth. Reject under-18. Hash the password with Argon2id; never store or log the plain password.
- **Login is mandatory every time:** whenever anyone opens the game, the server asks for username + password. No game, lobby, wallet or socket connection is allowed without a valid login. The Socket.IO handshake must verify the token and reject unauthenticated connections.
- Use a generic error ("Invalid username or password") so attackers cannot tell which one was wrong.
- Sessions: short-lived JWT access token + refresh token in an httpOnly cookie. Logging out (or token expiry) returns the player to the login screen. Add "log out from all devices".
- Do NOT auto-login from saved credentials on the server side; the player must sign in again after logout or session expiry. (Browser password managers are the player's choice.)
- Protection: rate limit per IP and per username, lockout for 15 minutes after 5 failed attempts, change-password endpoint (requires current password), roles (player/admin). **There is exactly ONE admin account**, created only by a server-side seed/CLI command using `ADMIN_USERNAME` and `ADMIN_PASSWORD` from env. The public register endpoint can only ever create `player` accounts

(ignore any `role` field sent by the client). No endpoint can promote a user to admin, and a second admin cannot be created (enforce with a DB unique partial index on role = 'admin' plus a server check).

- Tests for register, login, wrong password, locked account, expired token, unauthenticated socket, under-18, duplicate username, and brute-force attempts.

PHASE 4: Wallet & Ledger (most critical)

- `transfer(from, to, amount, type, idempotencyKey)` in one transaction using `SELECT ... FOR UPDATE`.
- Operations: deposit credit, withdraw hold/complete/release, buy ticket, payout, house fee, refund.
- Replaying the same idempotency key returns the original result.
- Concurrency test: 1000 parallel purchases on one wallet must never overdraw or double-spend.
- Reconciliation job: wallet balance must equal ledger sum.

PHASE 5: Payments (simple)

Define a `PaymentProvider` interface (initiateDeposit, verifyWebhook, initiatePayout). Build a mock provider first. Then one real local provider (e.g. Chapa or Telebirr) behind the same interface. Webhooks: verify signature, idempotent processing. Credentials only in env.

Admin approval is required for EVERY deposit and EVERY withdrawal (no threshold, no exceptions):

- **Deposit request:** the player submits an amount (and payment reference/screenshot/transaction ID). Status = `PENDING`. The wallet is NOT credited yet. An admin reviews it and either **approves** (the server then credits the wallet via a ledger transaction, once only) or **rejects** (with a mandatory reason; no credit).
- **Withdrawal request:** the player submits an amount and payout account. The server immediately **holds** that amount (balance -> locked) so it cannot be wagered. Status = `PENDING`. An admin **approves** (hold is completed and the payout is sent or marked paid) or **rejects** (with a mandatory reason; the hold is released back to the player's balance).
- Status machine: `PENDING -> APPROVED | REJECTED` (final states cannot change; a second approve/reject click must be safely ignored, i.e. idempotent). Only users with role `admin` can approve or reject; the server checks this on every call.
- Each decision records who decided, when, and why (audit log). The player gets a notification of the result.
- Withdrawal rules still apply: min/max amount, cannot withdraw more than the available balance, and a player can have only one pending withdrawal at a time.
- Tests: player cannot approve their own request, a non-admin gets 403, double-approve credits only once, reject releases the hold, concurrent approve/reject on the same request produces one result.

PHASE 6: Real-Time Game Server

- Round states: OPEN (buying tickets) -> LOCKED -> DRAWING -> SETTLED or CANCELLED.
- Rooms with ticket price, pattern, min players, house fee (basis points).
- Server timer draws a ball every 3-5 seconds (configurable); clients only display.
- Socket events: `room:state`, `ticket:bought`, `draw:number`, `claim:submitted`, `round:won`, `round:settled`.
- Auto-daub on by default. Player presses BINGO; server validates; false claims are rate-limited. Ties split the pot.
- If min players not reached, cancel the round and refund everyone.
- Payout in one transaction: pot -> house fee -> winner share(s) -> ledger -> mark SETTLED -> reveal serverSeed.
- Reconnect: a returning player gets full state (tickets, draws so far, countdown). Persist round state in DB so a server restart resumes the round.
- Tests: duplicate claims, simultaneous claims, restart mid-round.

Shared live rounds and late joiners (applies to Phase 6 and 7)

- **One shared game per room for everyone.** A player in Addis Ababa and a player in Gondar (or anywhere) who are in the same room join the same Socket.IO room and see the same round at the same time. There is one round per room on the server, never one per player, and the server broadcasts each ball to all connected players together.
- **Same start minute for everyone.** The server schedules each round with a single absolute start time (UTC timestamp, e.g. `startsAt`). Clients show a countdown computed from the server clock, not the phone clock: on connect, the client syncs by asking the server for its time and computing an offset, and re-syncs periodically. Rounds start automatically on a fixed cadence (configurable, e.g. every 2 minutes, at :00 seconds) so everyone sees "next game starts in mm:ss" and the draw begins at the same moment for all. Display times in the player's local timezone (Africa/Addis_Ababa by default); store and compute in UTC.
- **Ticket buying closes at lock time.** Tickets can be bought only while the round is `OPEN`, before `startsAt`. When the countdown hits zero, the server locks the round (no more tickets, no more card changes) and starts drawing.
- **Joining while a game is already running:** if a player logs in or opens a room after the round has started (`LOCKED` / `DRAWING` / `CLAIMING`), they:
  - are placed in **spectator mode** for the running round: they receive the live state (balls drawn so far, current ball, number of players, prize pool) and watch it live,
  - cannot buy a ticket or claim in that round (the server rejects it),
  - see a clear message such as "A game is in progress. You can join the next game when it ends." with the countdown to the next round,

    - automatically get the **next round's** ticket-buying screen as soon as the current round is `SETTLED` or `CANCELLED` and the next round is `OPEN`. No refresh needed (server pushes `room:state`).

- **Late joiners never affect fairness or money:** spectators have no tickets, no wallet changes, and do not count toward the player count or prize pool of the running round.
- **Reconnect rule:** a player who was in the round and reconnects (network drop) gets their tickets and full state back and stays a participant, not a spectator.
- **Tests:** two clients in different simulated timezones with skewed clocks receive the same `draw:number` events and the same `startsAt`; a client connecting mid-round is spectator-only, is rejected if it tries to buy a ticket or claim, and receives the next round's OPEN state automatically after settlement; a participant who reconnects keeps their tickets.

PHASE 7: Player Web App (mobile-first)

The first screen is always the **Login screen** (username + password, "Create account" link, show/hide password). Nothing else is reachable until login succeeds; if the session expires, redirect back to Login.
Screens: register (username, password, confirm password, date of birth), login, lobby (rooms, prize pool, countdown), card picker, live game (big current ball, last 5 balls, 1-75 board, my cards, BINGO button, sound toggle), result, wallet (deposit/withdraw/history), responsible-gaming settings, fairness verifier page.
Amharic + English, works on low-end phones and slow networks, auto-reconnect, never show a stale balance.

Live game screen layout (cartela + called numbers side by side)

- **Side by side:** the player's cartela (bingo card) and the board of called numbers are shown next to each other on the same screen, so the player can compare them at a glance without switching tabs.
    - **Left:** the player's cartela(s) (5x5, B-I-N-G-O header, center FREE), with called numbers auto-daubed/highlighted.
    - **Right:** the **master board of all numbers 1-75**, arranged in five rows B (1-15), I (16-30), N (31-45), G (46-60), O (61-75). Every number already called is clearly highlighted (filled color + bold), uncalled numbers are dimmed, and the **latest ball** has a distinct pulse/outline.
- **Overall called numbers always clearly visible:** the full list of called numbers is never hidden behind a menu or scroll. Keep it permanently on screen in two ways: (1) the master 1-75 board above, and (2) a "Called so far" strip showing the numbers in the order they were drawn (most recent first), with the count (e.g. "23 / 75").
- **Top area:** big current ball (letter + number, e.g. "G 52"), the last 5 balls, round countdown, prize pool, number of players.
- **Bottom area:** the large BINGO button and sound/voice toggle.

- **Small phone (narrow portrait screens):** keep them side by side using a compact two-column layout (cartela ~55% width, master board ~45% width, small but readable cells, minimum touch-safe sizes); do not stack them vertically so that one disappears below the fold. If the player holds several cartelas, show a horizontal swipe/tab switcher for cartelas while the master board stays fixed on the right. On wide screens (tablet/desktop) use a larger two-panel layout.
- **Spectators** (late joiners) see the same layout, with the cartela panel replaced by an info message ("Game in progress - join the next game").
- **Accessibility:** do not rely on color alone for "called" (also use bold/check mark); WCAG AA contrast; large numerals; works in dark and light themes.
- **Tests (Playwright, mobile viewport e.g. 360x640):** cartela and master board are both visible without scrolling; a `draw:number` event highlights the number on both the cartela (if present) and the master board and adds it to the "Called so far" strip; spectators see the master board and strip.

PHASE 8: Responsible Gaming

Deposit limits (decrease takes effect immediately, increase after a delay), self-exclusion enforced server-side on every API and socket action, session-time reminders, show odds/prize/house fee on every room, link to help resources.

PHASE 9: Simple Admin Panel

Dashboard (deposits, withdrawals, GGR, active players), create/edit rooms, cancel round with refunds, **player management** (see below), **deposit approval queue and withdrawal approval queue** (list of pending requests with player, amount, reference, time; Approve and Reject buttons; Reject requires a reason; live count of pending items; history of past decisions; real-time or auto-refresh when new requests arrive), manual adjustment with mandatory reason (logged), audit log viewer. Admin requires 2FA.

Admin powers over players, and a hidden single admin

- **Search and list players** (by username, status, online now).
- **Kick:** the admin can remove an online player immediately. The server disconnects all of that player's sockets, revokes their refresh tokens, and the player returns to the Login screen. A kick is temporary (they can log in again) and is logged. If the player is in a running round, their tickets stay valid and any win is still paid to their wallet.
- **Block:** the admin blocks a player with a mandatory reason. The player's status becomes `BLOCKED`: all sessions are revoked, sockets are disconnected, login is refused with a clear message ("Your account is blocked, contact support"), and no API or socket action works. Deposits, withdrawals and ticket purchases are rejected. The player's wallet balance is kept untouched and visible to the admin.
- **Unblock:** the admin restores the account to `ACTIVE`, and the player can log in again. Every kick, block and unblock records admin, player, time and reason in the audit log, and the player gets a

notification where possible.

- All of this is enforced server-side on every API call and on every socket event, not only at login.
- **The admin is invisible to players:**
  - The player app has no admin link, admin route, or admin code bundle. The admin panel is a separate app (`admin/`) served on a separate path or subdomain that is not linked from the player app.
  - The admin never appears in any player-facing list, lobby, leaderboard, chat, winners list, search, or API response.
  - The admin account is not a player: it has no wallet, cannot buy tickets, and cannot win.
  - If someone enters the admin username on the player login screen, the player app gives the same generic "Invalid username or password" error, and player endpoints refuse admin credentials. The admin signs in only through the admin panel login.
  - Admin API routes return 404 (not 403) to non-admins so their existence is not revealed.
  - Admin login is rate-limited, requires 2FA, and uses a short session.
- Tests: non-admin cannot kick/block/unblock; blocked user cannot log in or use an existing token or socket; kick disconnects live sockets; unblock restores access; a second admin cannot be created; admin never shows up in any player API; admin credentials fail on the player login endpoint.

PHASE 9.5: Recommended Additions (small but important)

1. **Betting limits:** per-room min/max cards per player per round (e.g. 1-5), set by admin. The server enforces it. Show the prize pool and house fee clearly before purchase.
2. **Clear prize math on screen:** show `pot = tickets x price`, house fee %, and prize = pot - fee. Rounding rule: integer minor units; any remainder from splitting goes to the house (or the first winner), documented and tested.
3. **Terms & age consent:** registration requires ticking "I am 18+ and accept the Terms". Store the timestamp and terms version. Add a Terms page and a Responsible Gaming page.
4. **Round history for players:** "My games" (tickets, result, win/loss) and "Past rounds" (draw sequence, revealed seed, verify button) so results are transparent.
5. **Notifications:** in-app messages (and optional Telegram/SMS later) for deposit approved/rejected, withdrawal approved/rejected, blocked/unblocked, win.
6. **Admin alerts:** a badge and a sound/toast on the admin panel when a new deposit or withdrawal request arrives; show the oldest pending request first; a daily summary report (deposits, withdrawals, GGR, payouts).
7. **Deposit proof:** deposit request accepts a payment reference/transaction ID and an optional screenshot (validate file type and size, store safely, show it to the admin). Reject duplicate transaction IDs so one payment cannot be claimed twice.
8. **Anti-abuse basics:** limit one active session per account (a new login kicks the old session, configurable), rate-limit ticket buying and claims, and flag accounts sharing the same device or IP

for the admin to review (flag only, do not auto-ban).

9. **Currency and formatting:** default currency ETB, amounts shown with proper formatting (e.g. 1,250.00 Birr), Ethiopian number words in Amharic UI where relevant.
10. **Operations:** automatic daily database backups, a documented restore test, HTTPS only, secrets only in env, error logging, and a clear "maintenance mode" switch the admin can turn on to stop new rounds while letting the current round finish.
11. **Stuck-round safety:** if a round cannot continue (server fault), the system cancels it and refunds all tickets automatically, logged and notified.
12. **Demo/practice mode (optional flag):** free play-money rooms so new players can learn without risk; play-money balance is completely separate from real wallets.

PHASE 10: Final Checks & Release

Playwright e2e (register -> deposit (mock) -> buy ticket -> watch draw -> claim -> payout -> withdraw). Docs: README (5-minute setup), FAIRNESS.md, COMPLIANCE.md, `.env.example`. CI with lint, typecheck, test, build.

ACCEPTANCE CHECKLIST

- [ ] A full round plays end to end with correct payout
- [ ] A finished round verifies independently (provably fair)
- [ ] Ledger reconciles to zero drift after concurrency tests
- [ ] No double-spend or double-payout
- [ ] Under-18 and self-excluded users blocked on API and sockets
- [ ] Every deposit and withdrawal stays PENDING until an admin approves or rejects it; money moves only after the decision, exactly once
- [ ] Players in different cities see the same live round and the same start time; mid-round joiners spectate and can buy tickets only for the next round
- [ ] Card limits per round, prize math and rounding are enforced and tested
- [ ] Duplicate deposit transaction IDs are rejected
- [ ] Maintenance mode and stuck-round auto-refund work
- [ ] Only one admin exists; the admin is hidden from players, and admin routes return 404 to non-admins
- [ ] Admin can kick, block and unblock players, enforced immediately on API and sockets
- [ ] Every visit requires username + password; unauthenticated API and socket access is rejected
- [ ] Restart mid-round resumes safely
- [ ] Amharic + English UI works on a low-end phone
- [ ] Tests pass; 100% coverage on engine and ledger

HOW TO RUN

Start with: "Read ROLE & RULES and TECH STACK, confirm understanding, then do PHASE 1 only. Stop and show me the test results and what's next." Repeat for each phase.